Sistemas de Informações

Arthur Parente Passos
Cassiano Gama Lemos
Pedro Lucas Sousa Chaves

RELATÓRIO TÉCNICO:
SISTEMA DE GERENCIAMENTO DE TRANSPORTE RODOVIÁRIO

Marabá – PA
2026

Arthur Parente Passos
Cassiano Gama Lemos
Pedro Lucas Sousa Chaves

SISTEMA DE GERENCIAMENTO DE TRANSPORTE RODOVIÁRIO

Relatório técnico apresentado à disciplina de Estruturas de Dados I, do curso de Sistemas de Informações, da como requisito parcial de avaliação.

Professor: ADAM DREYTON FERREIRA DOS SANTOS.

Marabá – PA
2026

Resumo

Este relatório apresenta o desenvolvimento de um sistema de gerenciamento de transporte rodoviário implementado na linguagem Java, com foco na utilização e aplicação das estruturas de dados ArrayList e LinkedList.

O sistema tem como finalidade administrar viagens, ônibus, passageiros, passagens vendidas e listas de espera, permitindo operações como cadastro, consulta, cancelamento, venda e geração de relatórios.

A implementação utiliza ArrayList para o armazenamento de viagens e passagens comercializadas, enquanto LinkedList é empregada na organização da fila de espera dos passageiros.

São apresentados os fundamentos da implementação, as regras de negócio, o relacionamento entre as classes, as justificativas para a escolha das estruturas de dados, as complexidades das operações, os algoritmos empregados, os formatos de entrada e saída e o manual de utilização. Também são identificadas limitações e inconsistências presentes na implementação analisada.

Palavras-chave: Java. Estruturas de dados. ArrayList. LinkedList. Transporte rodoviário.

Sumário

1 INTRODUÇÃO 7
1.1 Objetivo geral 7
1.2 Objetivos específicos 7

2 IDENTIFICAÇÃO DA EQUIPE E DO DOMÍNIO 8
2.1 Identificação da equipe 8
2.2 Domínio da aplicação 8
2.3 Entidades principais 8

3 VISÃO GERAL DO SISTEMA E REGRAS DE NEGÓCIO 9
3.1 Visão geral 9
3.2 Regras de negócio implementadas 9
3.2.1 Cadastro de viagens 9
3.2.2 Capacidade do ônibus 9
3.2.3 Venda de passagens 9
3.2.4 Tipos de passagem 10
3.2.5 Cancelamento de passagens 10
3.2.6 Remoção de viagens 10
3.2.7 Consulta e relatórios 10

4 DIAGRAMA DE CLASSES DA IMPLEMENTAÇÃO 11
4.1 Representação UML 11
4.2 Descrição das classes 12
4.3 Relacionamentos 13

5 JUSTIFICATIVA DA ESCOLHA DAS ESTRUTURAS DE DADOS (RC3) 14
5.1 ArrayList 14
5.1.1 Coleção de viagens 14
5.1.2 Coleção de passagens vendidas 14
5.2 LinkedList 14
5.2.1 Coleção de espera 15
5.3 Comparação entre as estruturas 15
5.4 Análise das operações e complexidades 15
5.4.1 Operações da classe Empresa 16
5.4.2 Operações da classe Viagem 16
5.5 Justificativa final da escolha 16

6 DESCRIÇÃO DOS ALGORITMOS (RC5) 18
6.1 Busca sequencial de viagens 18
6.2 Verificação de poltronas disponíveis 18
6.3 Gerenciamento FIFO da lista de espera 19
6.4 Cálculo do valor da passagem 19
6.5 Cálculo da receita 20
6.6 Cálculo da ocupação 20

7 FORMATO DOS ARQUIVOS DE ENTRADA E SAÍDA 21
7.1 Arquivo de entrada 21
7.1.1 Estrutura de uma viagem 21
7.1.2 Estrutura de uma passagem 21
7.1.3 Exemplo de arquivo de entrada 21
7.2 Arquivos de saída 22
7.3 Persistência dos dados 22

8 MANUAL DE UTILIZAÇÃO 23
8.1 Requisitos necessários 23
8.2 Organização dos arquivos 23
8.3 Compilação 24
8.4 Execução 24
8.5 Menu principal 24
8.6 Opção 1 – Listar viagens 24
8.7 Opção 2 – Detalhar viagem 25
8.8 Opção 3 – Cadastrar viagem 25
8.9 Opção 4 – Vender passagem 26
8.10 Opção 5 – Remover viagem 27
8.11 Opção 6 – Cancelar passagem 27
8.12 Opção 7 – Buscar viagem 27
8.13 Opção 8 – Listar viagens de um dia 28
8.14 Opção 9 – Gerar relatórios 28
8.15 Opção 10 – Salvar dados 28
8.16 Opção 11 – Sair 28

9 LIMITAÇÕES CONHECIDAS 30
9.1 Limitações funcionais 30
9.2 Limitações estruturais 30
9.3 Possíveis melhorias 30

10 DIVISÃO DE TAREFAS ENTRE OS MEMBROS 32

11 CONSIDERAÇÕES FINAIS 33

REFERÊNCIAS 34

1 INTRODUÇÃO

O gerenciamento de informações é uma necessidade fundamental em sistemas computacionais, especialmente quando existe a necessidade de armazenar, consultar, modificar e remover dados de maneira organizada.

No contexto do transporte rodoviário, o controle de viagens, ônibus, passageiros e passagens exige uma estrutura capaz de administrar diferentes entidades e suas relações.

Diante dessa necessidade, foi desenvolvido um sistema utilizando a linguagem Java, com aplicação das estruturas de dados ArrayList e LinkedList.

O projeto busca demonstrar, na prática, como diferentes estruturas podem ser utilizadas de acordo com as características das operações realizadas pelo sistema.

1.1 Objetivo geral

Desenvolver um sistema de gerenciamento de transporte rodoviário utilizando estruturas de dados dinâmicas em Java, aplicando os conceitos de ArrayList e LinkedList para administrar viagens, passagens e listas de espera.

1.2 Objetivos específicos

- Implementar o cadastro e gerenciamento de viagens.
- Controlar os dados dos ônibus e passageiros.
- Realizar a venda e o cancelamento de passagens.
- Implementar uma fila de espera para passageiros.
- Aplicar ArrayList e LinkedList em diferentes contextos.
- Analisar as complexidades das operações.
- Permitir a leitura e gravação de informações em arquivos externos.
- Gerar relatórios de ocupação e manifestos de passageiros.

2 IDENTIFICAÇÃO DA EQUIPE E DO DOMÍNIO

2.1 Identificação da equipe

O projeto foi desenvolvido pelos integrantes identificados na folha de rosto deste documento, sob orientação do professor ADAM DREYTON FERREIRA DOS SANTOS, na disciplina de Estruturas de Dados I.

2.2 Domínio da aplicação

O domínio escolhido para o desenvolvimento do sistema é o gerenciamento de transporte rodoviário.

A aplicação representa uma empresa responsável pela administração de viagens, veículos, passageiros e comercialização de passagens.

O sistema considera a existência de diferentes viagens, cada uma associada a um ônibus específico, com capacidade limitada de passageiros.

Quando uma viagem possui disponibilidade, o passageiro pode adquirir uma passagem e ocupar uma poltrona.

Quando não existe disponibilidade, o passageiro pode ser encaminhado para uma lista de espera, cujo atendimento ocorre conforme a ordem de entrada.

2.3 Entidades principais

As principais entidades identificadas no domínio são:

- **Empresa**: representa a empresa responsável pelo gerenciamento das viagens.
- **Viagem**: representa um deslocamento com origem, destino, data, horário e valor-base.
- **Ônibus**: representa o veículo utilizado na realização de uma viagem.
- **Passageiro**: armazena os dados de identificação do passageiro.
- **Passagem**: representa a aquisição de um assento por um passageiro.
- **TipoPassagem**: enumeração que identifica as modalidades inteira, estudante e idoso.

3 VISÃO GERAL DO SISTEMA E REGRAS DE NEGÓCIO

3.1 Visão geral

O sistema funciona por meio de uma interface de terminal, na qual o usuário interage com um menu de opções.

A classe Empresa mantém a coleção de viagens cadastradas. Cada objeto Viagem mantém duas coleções principais: uma para passagens vendidas e outra para passageiros em espera.

A classe Viagem também concentra as principais operações de gerenciamento, como venda, cancelamento, identificação de poltronas livres, cálculo de receita e ocupação.

O sistema utiliza arquivos de texto para carregar e armazenar informações, permitindo a manutenção dos dados entre execuções, embora a implementação apresente limitações nessa persistência.

3.2 Regras de negócio implementadas

As regras de negócio identificadas no código são apresentadas a seguir.

3.2.1 Cadastro de viagens

Uma viagem é cadastrada com código identificador, origem, destino, data e horário de partida, valor-base e informações do ônibus.

3.2.2 Capacidade do ônibus

Cada ônibus possui uma capacidade definida por um valor inteiro.

O número de passagens vendidas não deve ultrapassar a capacidade do veículo.

3.2.3 Venda de passagens

Para efetuar a venda, o sistema verifica:

- Se a poltrona está dentro dos limites de capacidade do ônibus.
- Se a poltrona já está ocupada.
- Se o documento do passageiro já está associado a uma passagem vendida.
- Se existe disponibilidade de assentos.

Caso o ônibus esteja lotado, o passageiro é encaminhado à lista de espera.

3.2.4 Tipos de passagem

O sistema possui três modalidades:

- **INTEIRA**: pagamento integral do valor-base.
- **ESTUDANTE**: pagamento de 50% do valor-base.
- **IDOSO**: gratuidade para os dois primeiros beneficiários identificados como gratuitos na viagem; posteriormente, pagamento de 50% do valor-base.

A regra de gratuidade é implementada pelo método `calcularValor()` da classe Passagem, considerando a quantidade de passagens gratuitas já registradas.

3.2.5 Cancelamento de passagens

O cancelamento ocorre por meio do número da poltrona.

Quando uma passagem é cancelada, o sistema verifica se existe alguém aguardando na fila.

Caso exista, o primeiro passageiro da fila é removido e associado à poltrona liberada.

O valor da nova passagem é recalculado considerando a regra de gratuidade para idosos.

3.2.6 Remoção de viagens

Uma viagem somente pode ser removida quando não possui passagens vendidas.

Essa restrição impede a exclusão de viagens que ainda possuem passageiros associados.

3.2.7 Consulta e relatórios

O sistema permite consultar viagens, listar passageiros, visualizar a fila de espera, consultar viagens por data e calcular indicadores de ocupação e receita.

4 DIAGRAMA DE CLASSES DA IMPLEMENTAÇÃO

4.1 Representação UML

O diagrama apresenta as classes identificadas na implementação e os relacionamentos entre seus objetos.

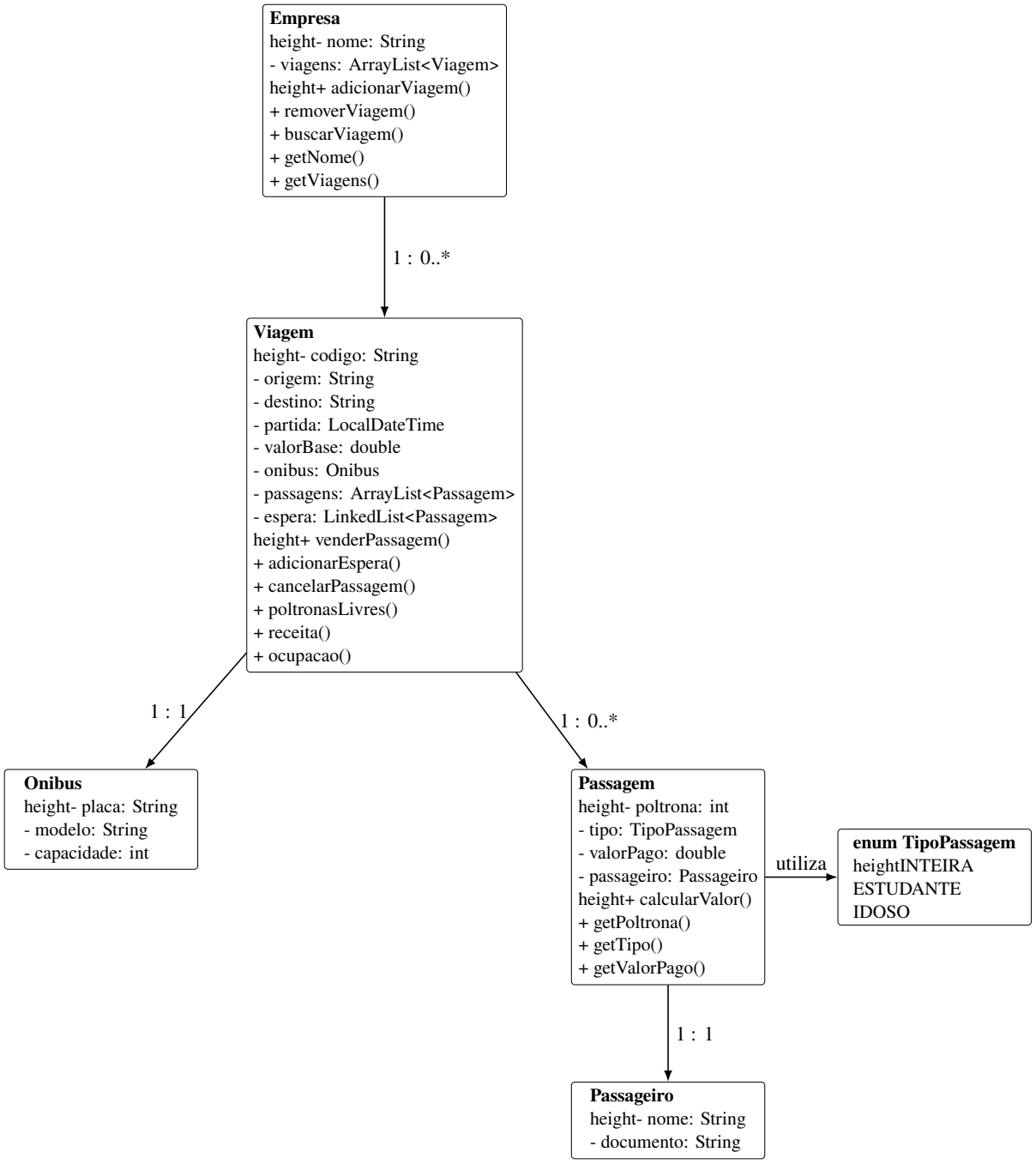


Figura 1 – Diagrama de classes do sistema

4.2 Descrição das classes

Tabela 1 – Responsabilidades das classes

Classe	**Responsabilidade**
Empresa	Armazenar o nome da empresa e administrar a coleção de viagens.

Classe	Responsabilidade
Viagem	Gerenciar os dados da viagem, ônibus, passagens vendidas e fila de espera.
Onibus	Armazenar placa, modelo e capacidade do veículo.
Passageiro	Armazenar nome e documento de identificação.
Passagem	Representar a passagem, a poltrona, o tipo, o valor pago e o passageiro.
TipoPassagem	Definir as modalidades de passagem disponíveis por meio de uma enumeração.

4.3 Relacionamentos

A classe Empresa possui uma coleção de objetos Viagem.

Cada Viagem está associada a um objeto Onibus e mantém coleções de objetos Passagem.

Cada Passagem possui uma referência para um objeto Passageiro e utiliza um valor da enumeração TipoPassagem.

As coleções de passagens vendidas e de espera são independentes, mas representam situações distintas dentro de uma mesma viagem.

5 JUSTIFICATIVA DA ESCOLHA DAS ESTRUTURAS DE DADOS (RC3)

5.1 ArrayList

ArrayList é uma implementação da interface List baseada em um vetor redimensionável.

Essa estrutura permite acesso direto aos elementos por índice, apresentando complexidade constante para operações de acesso, em condições normais.

No projeto, ArrayList foi utilizado em duas coleções principais.

5.1.1 Coleção de viagens

Na classe Empresa:

```
1 private ArrayList<Viagem> viagens;
```

A escolha é adequada porque o sistema realiza principalmente inserções, consultas sequenciais e listagens das viagens.

O acesso por índice não é utilizado diretamente com frequência, mas a estrutura oferece facilidade de implementação e bom desempenho para percorrer os registros.

5.1.2 Coleção de passagens vendidas

Na classe Viagem:

```
1 private ArrayList<Passagem> passagens;
```

A coleção armazena as passagens comercializadas.

O sistema realiza operações de consulta, verificação de duplicidade, cálculo de receita, cancelamento e listagem.

A utilização de ArrayList permite percorrer todos os elementos com facilidade e acessar passagens por índice.

5.2 LinkedList

LinkedList é uma estrutura de dados baseada em uma lista duplamente encadeada.

Diferentemente do ArrayList, seus elementos são armazenados em nós que possuem referências para os elementos anterior e posterior.

No projeto, a estrutura é utilizada para representar a fila de espera.

5.2.1 Coleção de espera

Na classe Viagem:

```
private LinkedList<Passagem> espera;
```

A escolha está relacionada ao comportamento FIFO (First In, First Out), no qual o primeiro elemento inserido é o primeiro a ser removido.

O método adicionarEspera() utiliza:

```
public void adicionarEspera(Passagem p){
    espera.add(p);
}
```

Para retirar o primeiro passageiro:

```
Passagem proximaEspera = espera.removeFirst();
```

A remoção do primeiro elemento possui complexidade O(1), pois a estrutura mantém referência para o início da lista.

5.3 Comparação entre as estruturas

Tabela 2 – Comparação entre ArrayList e LinkedList

Operação	**ArrayList**	**LinkedList**
Acesso por índice	O(1)	O(n)
Inserção no final	O(1) amortizado	O(1)
Remoção no início	O(n)	O(1)
Busca por valor	O(n)	O(n)
Remoção por índice	O(n)	O(n)
Percurso completo	O(n)	O(n)

Fonte: Elaborado pela equipe (2026).

5.4 Análise das operações e complexidades

Para facilitar a análise, serão utilizadas as seguintes variáveis:

- V: quantidade de viagens cadastradas.
- P: quantidade de passagens vendidas em uma viagem.
- E: quantidade de passageiros na espera.
- C: capacidade total do ônibus.

5.4.1 Operações da classe Empresa

Tabela 3 – Complexidade das operações da classe Empresa

Método	Operação	Complexidade
adicionarViagem()	Inserção no final da coleção.	O(1) amortizado
removerViagem()	Busca e remoção do objeto.	O(V)
buscarViagem()	Percurso sequencial por código.	O(V)
getViagens()	Retorno da referência da coleção.	O(1)

Fonte: Elaborado pela equipe (2026).

A operação removerViagem() possui complexidade O(V), considerando a busca e o deslocamento dos elementos necessários à remoção.

5.4.2 Operações da classe Viagem

Tabela 4 – Complexidade das operações da classe Viagem

Método	Operação	Complexidade
venderPassagem()	Verificação de poltrona e documento.	O(P)
adicionarEspera()	Inserção no final da LinkedList.	O(1)
cancelarPassagem()	Busca, remoção e eventual promoção da espera.	O(P+E)
poltronasLivres()	Comparação de cada poltrona com as passagens.	$O(C \cdot P)$
receita()	Soma dos valores das passagens.	O(P)
ocupacao()	Cálculo com base na quantidade de passagens.	O(1)

Fonte: Elaborado pela equipe (2026).

A operação poltronasLivres() apresenta complexidade $O(C \cdot P)$, pois percorre todas as poltronas e, para cada uma, verifica a existência de uma passagem correspondente.

O método cancelarPassagem() apresenta custo O(P+E) no pior caso, considerando a busca, a remoção da passagem e a verificação das passagens existentes para recalcular a gratuidade. A remoção do primeiro elemento da LinkedList é O(1).

5.5 Justificativa final da escolha

A utilização das duas estruturas permite adequar o armazenamento às características das operações.

O ArrayList foi escolhido para as viagens e passagens vendidas devido à facilidade de percurso, acesso por índice e inserção no final.

O LinkedList foi escolhido para a lista de espera devido à necessidade de inserir passageiros no final e removê-los pelo início, respeitando a ordem FIFO.

Essa separação demonstra a aplicação prática dos conceitos de estruturas de dados e a importância de considerar o custo das operações ao selecionar uma implementação.

6 DESCRIÇÃO DOS ALGORITMOS (RC5)

6.1 Busca sequencial de viagens

Localização: classe Empresa, método buscarViagem().

O algoritmo percorre sequencialmente a coleção de viagens até encontrar uma viagem cujo código seja igual ao código informado.

A comparação não diferencia letras maiúsculas e minúsculas, devido ao uso de equalsIgnoreCase().

Caso encontre o registro, retorna o objeto correspondente. Caso contrário, retorna null.

```
public Viagem buscarViagem(String codigo){
    for(Viagem v : viagens){
        if(v.getCodigo().equalsIgnoreCase(codigo)){
            return v;
        }
    }
    return null;
}
```

Complexidade:

$$O(V)$$

6.2 Verificação de poltronas disponíveis

Localização: classe Viagem, método poltronasLivres().

O algoritmo percorre todas as poltronas disponíveis no ônibus.

Para cada poltrona, verifica se existe uma passagem vendida com o mesmo número.

Quando não encontra uma passagem associada, adiciona o número à coleção de poltronas livres.

```
for(int i = 1; i <= onibus.getCapacidade(); i++){
    boolean ocupada = false;

    for(Passagem p : passagens){
        if(p.getPoltrona() == i){
            ocupada = true;
            break;
        }
```

```
9       }
10
11      if(!ocupada){
12          livres.add(i);
13      }
14  }
```

Complexidade:

$$O(C \cdot P)$$

6.3 Gerenciamento FIFO da lista de espera

Localização: classe Viagem, métodos adicionarEspera() e cancelarPassagem().

O algoritmo utiliza o comportamento FIFO (First In, First Out).

Quando um passageiro não consegue adquirir uma poltrona, ele é adicionado ao final da LinkedList.

Quando ocorre um cancelamento, o sistema remove o primeiro passageiro da espera, utilizando removeFirst().

A poltrona liberada é atribuída ao passageiro removido da fila.

```
1  espera.add(p);
2
3  Passagem proximaEspera = espera.removeFirst();
4
5  proximaEspera.setPoltrona(poltrona);
6
7  passagens.add(proximaEspera);
```

Complexidade das operações da fila:

$$Insercao = O(1)$$

$$Remocao = O(1)$$

6.4 Cálculo do valor da passagem

Localização: classe Passagem, método calcularValor().

O algoritmo verifica o tipo da passagem e determina o valor a ser pago.

Para passagens inteiras, o valor corresponde ao preço-base.

Para estudantes, é aplicado um desconto de 50%.

Para idosos, são consideradas duas gratuidades por viagem. Após esse limite, aplica-se um desconto de 50%.

Complexidade:

$$O(1)$$

6.5 Cálculo da receita

Localização: classe Viagem, método receita().

O algoritmo percorre todas as passagens vendidas e soma os valores pagos.

```
1 public double receita(){
2     double total = 0.0;
3
4     for(Passagem p : passagens){
5         total += p.getValorPago();
6     }
7
8     return total;
9 }
```

Complexidade:

$$O(P)$$

6.6 Cálculo da ocupação

Localização: classe Viagem, método ocupacao().

O algoritmo calcula a porcentagem de ocupação do ônibus dividindo a quantidade de passagens vendidas pela capacidade total do veículo e multiplicando o resultado por 100.

$$Ocupacao = \frac{Passagens\ vendidas}{Capacidade} \times 100$$

A implementação apresenta complexidade O(1).

7 FORMATO DOS ARQUIVOS DE ENTRADA E SAÍDA

7.1 Arquivo de entrada

O sistema utiliza o arquivo:

```
dados/viagens.txt
```

O arquivo contém informações da empresa, das viagens e das passagens já cadastradas.
A primeira linha deve representar o nome da empresa.
As linhas seguintes representam as viagens, separando os campos por ponto e vírgula.

7.1.1 Estrutura de uma viagem

O formato esperado pelo carregador é:

```
CODIGO;ORIGEM;DESTINO;DATA;VALOR;
PLACA;MODELO;CAPACIDADE;QUANTIDADE_PASSAGENS
```

A data utiliza o padrão:

```
dd/MM/yyyy HH:mm:ss
```

7.1.2 Estrutura de uma passagem

Após a linha de uma viagem, são esperadas tantas linhas de passagem quanto o número informado no campo QUANTIDADE$_{P}ASSAGENS$.

```
POLTRONA;NOME;DOCUMENTO;TIPO;VALOR_PAGO
```

7.1.3 Exemplo de arquivo de entrada

```
Empresa C.A.P Viagens
V001;Maraba;Belem;15/12/2026 08:00:00;150.00;ABC1234;Mercedes;40;2
1;Joao Silva;123456789;INTEIRA;150.00
2;Maria Souza;987654321;ESTUDANTE;75.00
V002;Maraba;Imperatriz;16/12/2026 09:00:00;100.00;DEF5678;Volvo;35;0
```

Observação técnica: o código de leitura apresentado possui um erro na identificação do documento do passageiro.

Atualmente, utiliza:

```
String documento = camposPassagem[1].trim();
```

O correto, considerando o formato definido, é:

```
String documento = camposPassagem[2].trim();
```

7.2 Arquivos de saída

O sistema prevê a geração de relatórios na pasta:

```
saidas/
```

Os arquivos previstos são:

- `ocupacao.txt`: apresenta informações de ocupação, receita e quantidade de passageiros em espera.
- `manifesto_CODIGO.txt`: apresenta as informações dos passageiros e das passagens de cada viagem.

7.3 Persistência dos dados

A persistência principal ocorre no arquivo:

```
dados/viagens.txt
```

O sistema utiliza o objeto PrintWriter para escrever as informações das viagens e passagens.

Entretanto, a implementação atual possui uma inconsistência na primeira linha gravada: é utilizada a representação da coleção de viagens no lugar do nome da empresa.

O trecho deveria utilizar:

```
writer.println(empresa.getNome());
```

Também é necessário garantir que a pasta `dados` exista antes da gravação.

A lista de espera não é armazenada no arquivo, portanto, os passageiros aguardando atendimento não são preservados após o encerramento do programa.

8 MANUAL DE UTILIZAÇÃO

8.1 Requisitos necessários

Para executar o sistema, são necessários:

- Java Development Kit (JDK).
- Ambiente de desenvolvimento ou terminal.
- Arquivos de código-fonte Java.
- Estrutura de diretórios adequada.

8.2 Organização dos arquivos

Uma estrutura recomendada para o projeto é:

```
ProjetoTransporte/
|
|-- src/
|   |-- Empresa.java
|   |-- Onibus.java
|   |-- Passageiro.java
|   |-- Passagem.java
|   |-- Viagem.java
|   |-- TipoPassagem.java
|   |-- SistemaTransporte.java
|
|-- dados/
|   |-- viagens.txt
|
|-- saidas/
|
|-- bin/
```

O nome SistemaTransporte.java é uma sugestão para a classe que contém o método main, pois o nome da classe principal não foi informado no código fornecido.

8.3 Compilação

Abra o terminal na pasta principal do projeto e execute:

```
javac -encoding UTF-8 -d bin src/*.java
```

Esse comando compila os arquivos Java e armazena os arquivos .class na pasta bin.

8.4 Execução

Após a compilação, execute:

```
java -cp bin SistemaTransporte
```

O nome da classe deverá corresponder à classe pública que contém o método main.

8.5 Menu principal

A interface principal disponibiliza as seguintes operações:

```
--- SISTEMA DE TRANSPORTE ---

1. Listar viagens principais com resumo
2. Detalhar uma viagem
3. Cadastrar nova viagem
4. Vender passagem
5. Remover uma viagem
6. Cancelar passagem
7. Buscar viagem pelo codigo
8. Listar viagens de um dia
10. Salvar dados em arquivo
11. Sair

Escolha uma opcao:
```

A opção 9 não aparece no menu apresentado, embora exista um bloco case 9 no código para geração de relatórios.

8.6 Opção 1 – Listar viagens

Apresenta todas as viagens cadastradas, com código, origem, destino, horário e percentual de ocupação.

Exemplo:

```
--- LISTA DE VIAGENS ---

Codigo: V001 | Maraba -> Belem
Partida: 15/12/2026 08:00:00
Ocupacao: 5.0%

Codigo: V002 | Maraba -> Imperatriz
Partida: 16/12/2026 09:00:00
Ocupacao: 0.0%
```

8.7 Opção 2 – Detalhar viagem

Solicita o código de uma viagem e apresenta as passagens vendidas e os passageiros na lista de espera.

Exemplo:

```
Digite o codigo da viagem:
V001

Viagem: V001 (Maraba -> Belem)

Passagens Vendidas (2):
Poltrona 1: Joao Silva [INTEIRA]
Poltrona 2: Maria Souza [ESTUDANTE]

Lista de Espera (0):
```

8.8 Opção 3 – Cadastrar viagem

Solicita as informações necessárias para criar uma nova viagem.

Os dados incluem código, origem, destino, data, horário, valor-base, placa, modelo e capacidade do ônibus.

Exemplo:

```
Codigo da viagem:
V003

Origem:
Maraba

```

```
Destino:
Tucurui

Data e hora:
20/12/2026 10:00:00

Valor base:
120.00

Placa:
XYZ1234

Modelo:
Mercedes

Capacidade:
40

Viagem cadastrada com sucesso!
```

8.9 Opção 4 – Vender passagem

Permite registrar uma nova passagem, solicitando o código da viagem, poltrona, nome, documento e tipo de passagem.

O sistema apresenta as poltronas livres e calcula o valor correspondente.

Exemplo:

```
Codigo da viagem:
V001

Poltronas livres: [3, 4, 5, 6]

Numero da poltrona desejada:
3

Nome do passageiro:
Carlos Lima

Documento:
111222333

```

```
Tipo:
INTEIRA

Passagem vendida com sucesso!
Valor: R$ 150.00
```

8.10 Opção 5 – Remover viagem

Permite excluir uma viagem somente quando não existem passagens vendidas.

Exemplo:

```
Codigo da viagem a remover:
V002

Viagem removida com sucesso!
```

8.11 Opção 6 – Cancelar passagem

Solicita o código da viagem e a poltrona da passagem que será cancelada.

Caso exista alguém aguardando, o primeiro passageiro da fila recebe a poltrona liberada.

Exemplo:

```
Codigo da viagem:
V001

Numero da poltrona a cancelar:
2

Passagem da poltrona 2
cancelada com sucesso.
```

8.12 Opção 7 – Buscar viagem

Realiza a busca de uma viagem por meio de seu código identificador.

Exemplo:

```
Digite o codigo da viagem:
V001

Encontrada: V001 | Maraba -> Belem
```

8.13 Opção 8 – Listar viagens de um dia

Permite filtrar as viagens de acordo com uma data específica.

Exemplo:

```
Digite a data desejada:
15/12/2026

--- VIAGENS DO DIA 15/12/2026 ---

Codigo: V001
Horario: 08:00
Maraba -> Belem
```

8.14 Opção 9 – Gerar relatórios

Embora não apareça no menu principal, existe uma implementação correspondente ao case 9.

Essa opção gera o relatório de ocupação e os manifestos individuais das viagens.

Exemplo de saída:

```
Relatorios gerados com sucesso
na pasta saidas/
```

8.15 Opção 10 – Salvar dados

Realiza a gravação das viagens e passagens no arquivo dados/viagens.txt.

Exemplo:

```
Dados salvos com sucesso
em dados/viagens.txt
```

8.16 Opção 11 – Sair

Encerra a execução do sistema.

O programa pergunta se o usuário deseja salvar os dados antes de sair.

```
Deseja salvar os dados antes de sair? (s/n)


s

```

```
5 Saindo...
```

Atenção: na implementação fornecida, responder S não executa efetivamente a gravação do arquivo. É necessário utilizar a opção 10 antes de sair.

9 LIMITAÇÕES CONHECIDAS

Durante a análise da implementação, foram identificadas algumas limitações que podem afetar o funcionamento do sistema.

9.1 Limitações funcionais

- **Persistência da fila de espera:** os passageiros aguardando atendimento não são armazenados no arquivo principal.
- **Validação de documentos:** a leitura atual utiliza o campo incorreto para identificar o documento do passageiro.
- **Validação de duplicidade:** a verificação do documento considera passagens vendidas, mas não impede necessariamente duplicidades na lista de espera.
- **Opção de saída:** o sistema pergunta se o usuário deseja salvar, mas não executa a gravação automaticamente.
- **Menu incompleto:** a opção 9 está implementada, porém não aparece no menu apresentado.
- **Capacidade zero:** o método ocupacao() trata a capacidade zero, mas o cálculo do relatório da opção 9 não apresenta a mesma proteção.
- **Tratamento de erros:** algumas operações de entrada podem gerar exceções quando os dados informados não correspondem ao formato esperado.

9.2 Limitações estruturais

A aplicação utiliza armazenamento em memória durante a execução, sem banco de dados.

A ausência de uma camada específica de persistência dificulta a manutenção e a expansão do sistema.

Também não existe um mecanismo de autenticação ou controle de permissões de usuários.

A coleção de viagens e as coleções de passagens são disponibilizadas diretamente pelos métodos getViagens() e getPassagens(), permitindo alterações externas sem passar pelas regras de negócio.

9.3 Possíveis melhorias

Como possibilidades de evolução, destacam-se:

- Implementação de persistência completa da lista de espera.
- Correção das inconsistências de leitura e gravação de arquivos.
- Inclusão de validações mais rigorosas para documentos e dados numéricos.
- Implementação de banco de dados.
- Separação da interface, regras de negócio e persistência.
- Criação de testes automatizados.
- Implementação de controle de usuários.
- Utilização de interfaces List em vez de expor diretamente ArrayList e LinkedList.
- Otimização da busca por poltronas livres, evitando a comparação repetida de todas as passagens.

10 DIVISÃO DE TAREFAS ENTRE OS MEMBROS

A divisão de tarefas deve representar a participação efetiva de cada integrante no desenvolvimento do projeto.

A tabela abaixo deverá ser preenchida pela equipe de acordo com as atividades realmente executadas.

Tabela 5 – Distribuição das atividades

Integrante	**Atividades desenvolvidas**
Integrante 1	Desenvolvimento das classes Empresa e Viagem, incluindo gerenciamento das coleções.
Integrante 2	Desenvolvimento das classes Passageiro, Passagem, Onibus e TipoPassagem.
Integrante 3	Implementação do menu, leitura e gravação de arquivos e geração de relatórios.
Integrante 4	Documentação, testes, análise de complexidade e elaboração do diagrama de classes.

Fonte: Elaborado pela equipe (2026).

Observação: a distribuição apresentada é apenas uma sugestão de organização. Os nomes e as atividades devem ser ajustados conforme a participação real dos integrantes.

11 CONSIDERAÇÕES FINAIS

O desenvolvimento do sistema de gerenciamento de transporte rodoviário permitiu aplicar conceitos fundamentais de estruturas de dados na linguagem Java.

A utilização de ArrayList demonstrou-se adequada para o armazenamento de viagens e passagens vendidas, principalmente devido à facilidade de percorrer os elementos e realizar operações de consulta.

Por outro lado, LinkedList foi utilizada na implementação da fila de espera, permitindo a inserção de novos passageiros no final e a remoção do primeiro elemento em tempo constante.

A aplicação também demonstrou como diferentes estruturas podem trabalhar em conjunto para representar situações distintas de um mesmo domínio.

A análise das operações permitiu identificar as complexidades envolvidas e compreender que a escolha de uma estrutura de dados deve considerar o comportamento esperado das operações realizadas.

Apesar de apresentar as funcionalidades principais propostas, a implementação possui limitações relacionadas à persistência, validação de dados e tratamento de exceções.

Esses pontos representam oportunidades de melhoria para versões futuras do sistema.

Conclui-se que o projeto atingiu seu propósito didático ao demonstrar a aplicação prática de ArrayList e LinkedList em um sistema desenvolvido com orientação a objetos.

Referências

[1] ORACLE. **Class ArrayList**. Java Platform, Standard Edition. Disponível em: https://docs.oracle.com/en/java/javase/. Acesso em: 6 nov. 2026.

[2] ORACLE. **Class LinkedList**. Java Platform, Standard Edition. Disponível em: https://docs.oracle.com/en/java/javase/. Acesso em: 6 nov. 2026.

[3] ORACLE. **The Collections Framework**. Java Documentation. Disponível em: https://docs.oracle.com/javase/tutorial/collections/. Acesso em: 6 nov. 2026.

[4] CORMEN, Thomas H. et al. **Algoritmos: teoria e prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.